

FLOWERBUD

PLANT CARE MADE SIMPLE



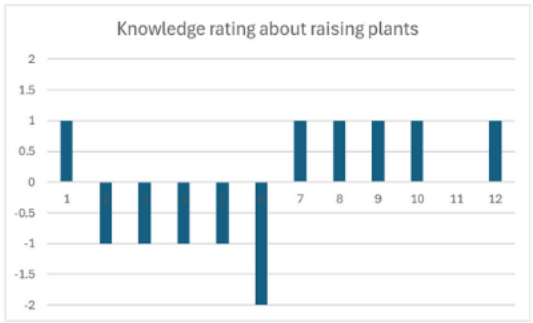


By Angie, Arife and Sharon

Problem to Solution

1. On a scale of -2 to 2, how interested are you in raising plants?







A poll of 2,000 people conducted in 2021 found over 50% of houseplant owners have killed an average of 5 plants accidentally



In a 2020 survey of 2,000 people 48% of participants said they were worried about keeping their houseplants alive

Little knowledge

Too forgetful

Tried but failed

No time

Lack of motivation

No space

- Watering reminders
- Plant recommendations based on user preferences (e.g. time and space availability)
- Detailed information about specific plants
- A journal to see progress and stay motivated

Key demographic

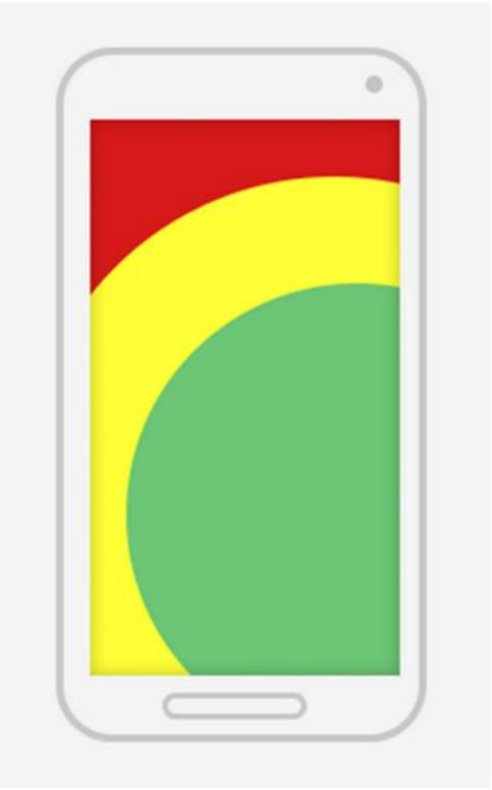
- Varying levels of experience
- Finds gardening difficult
- Busy
- Lives in an urban area
- Likes nature
- Lifestyle which is seemingly ill-suited to growing plants
- Seeking sense of responsibility or way to relieve stress
- Seeking a way to make indoor spaces more pleasant
- Looking for a hobby

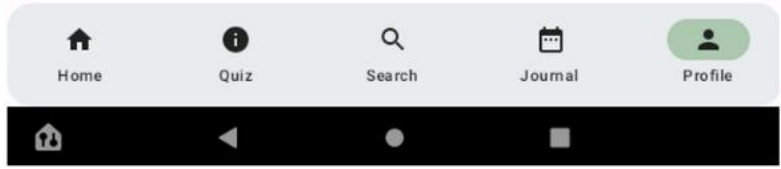
Design

Things that had to be considered:

- **Atmosphere**
  - Colour palette
- **UX and accessibility**
  - Contrast for visibility
  - Clear font for readability
  - Hand reach
  - Icons







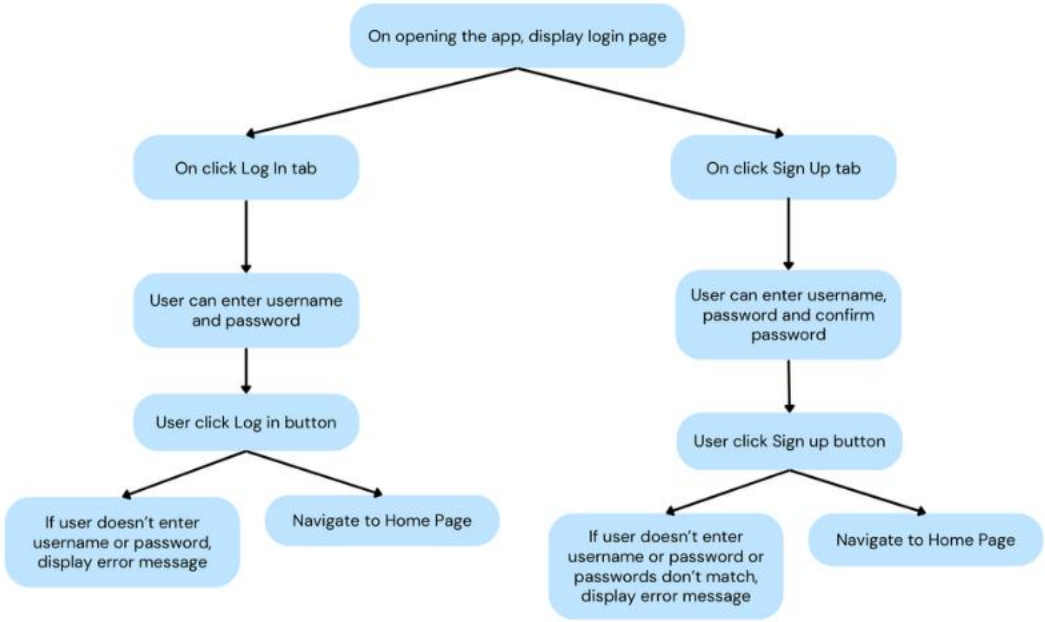
Log in page

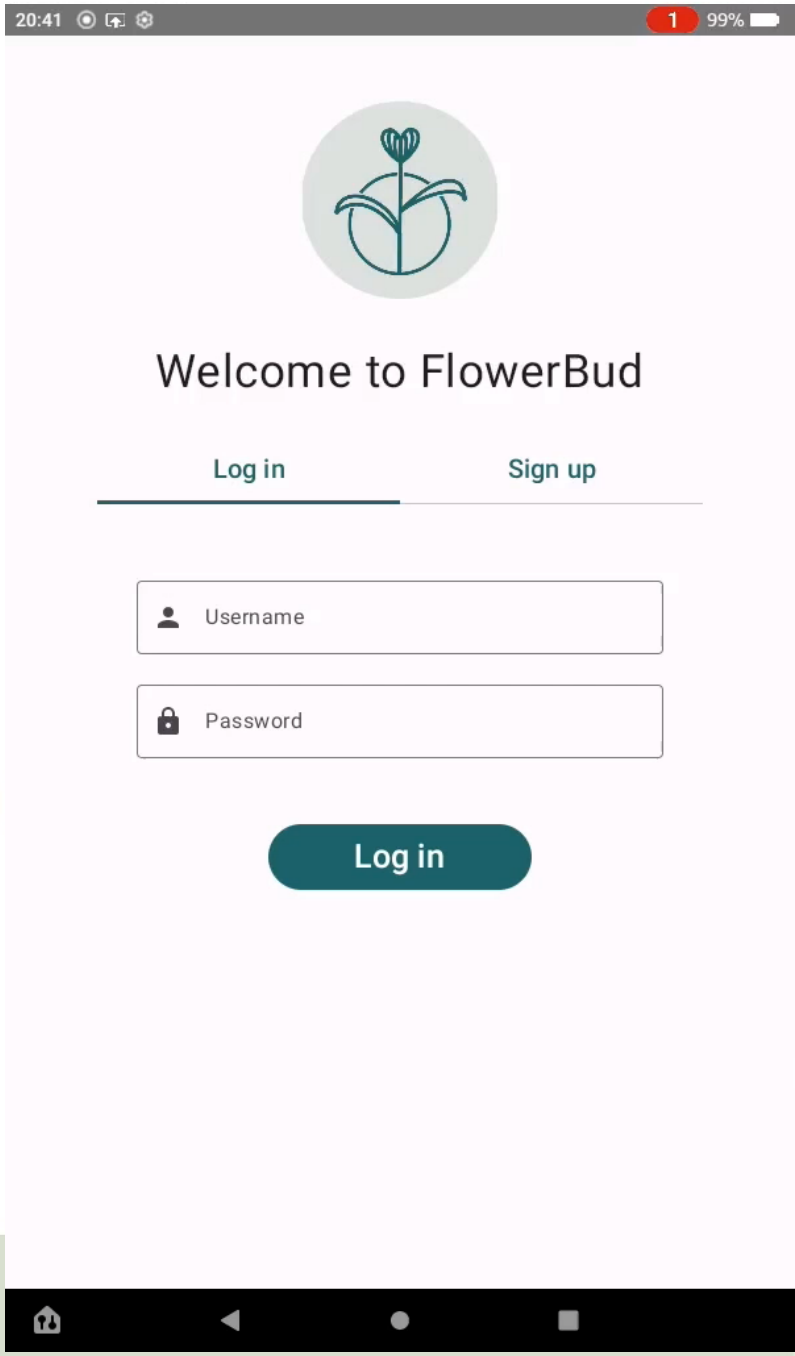
Key function:

- User has the choice to either sign up or log into their account

Design considerations:

- Simplicity of layout, conciseness of text
- Consistent colour scheme, font, etc.
- Easy readability: clear and understandable text, standard icons





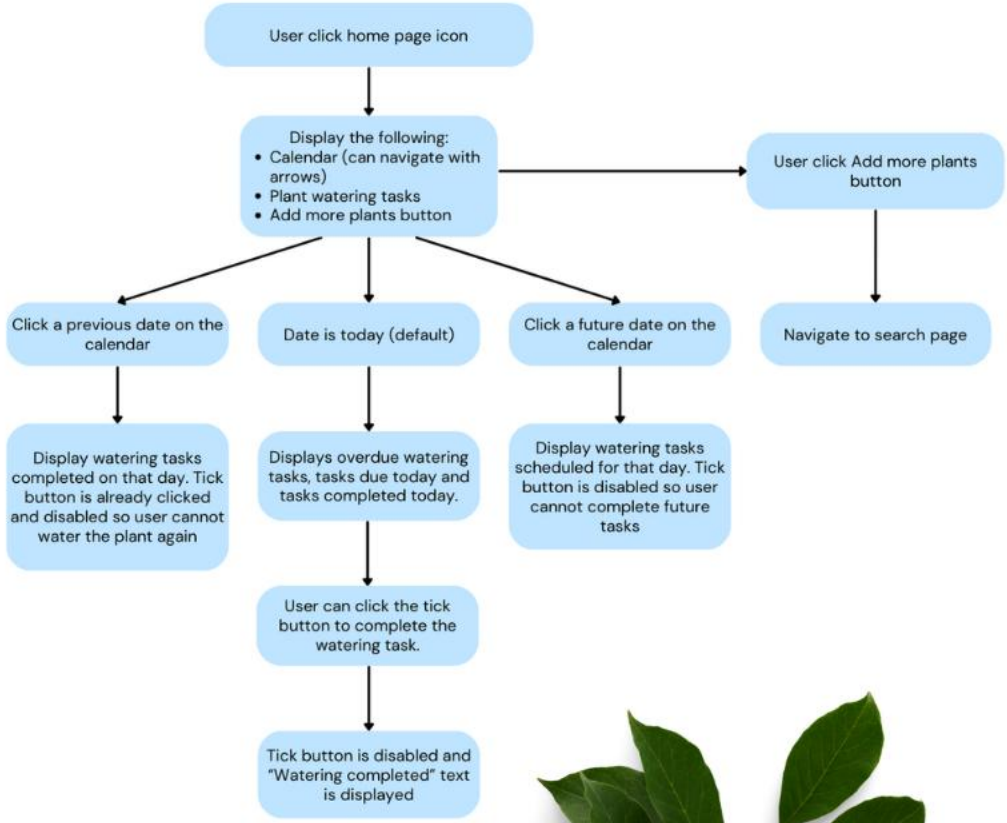
Home page

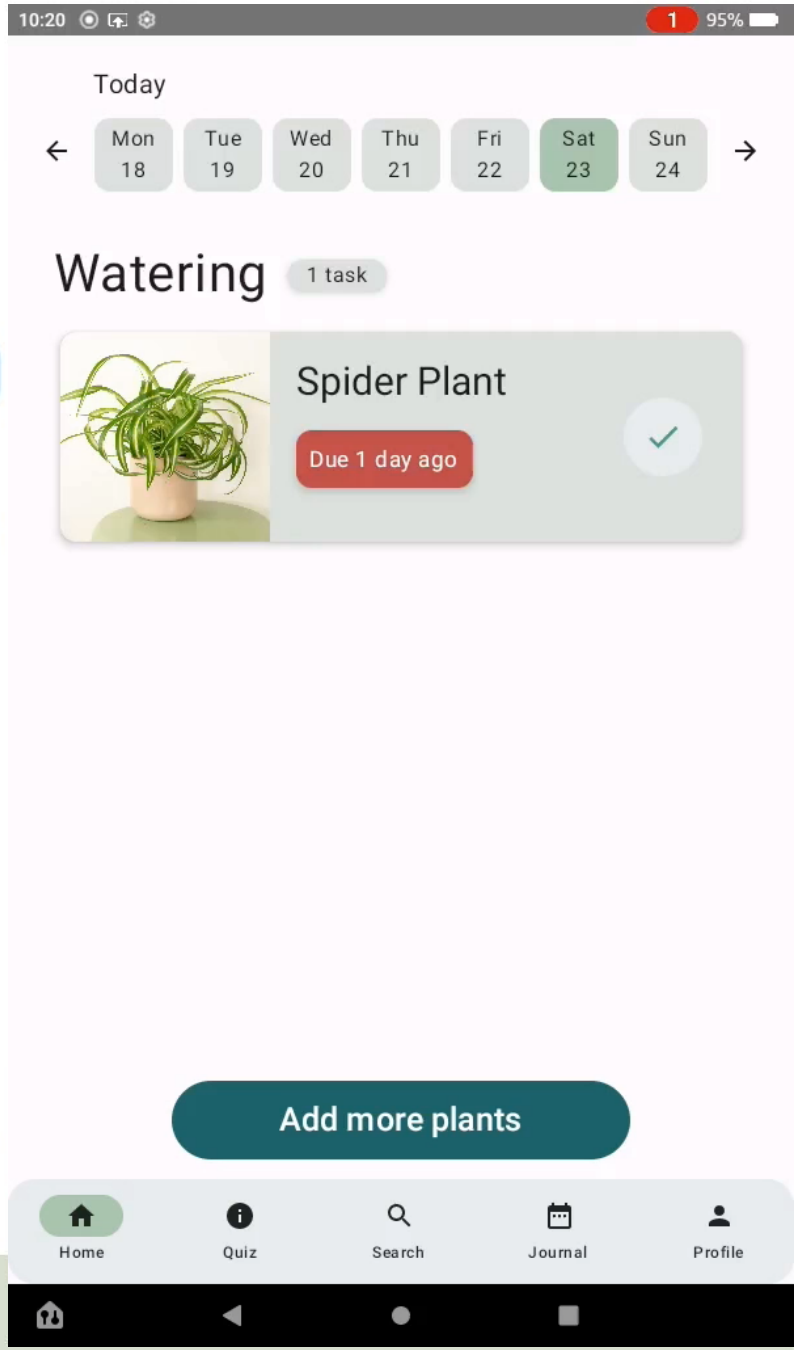
Key function:

- Displays the user's upcoming watering tasks and allows them to be checked off
- Allows users to navigate their calendar to see when the next watering is scheduled for

Design considerations:

- Simplicity of layout, conciseness of text
- Consistent colour scheme, font, etc.
- Easy to understand: good readability and overall usability





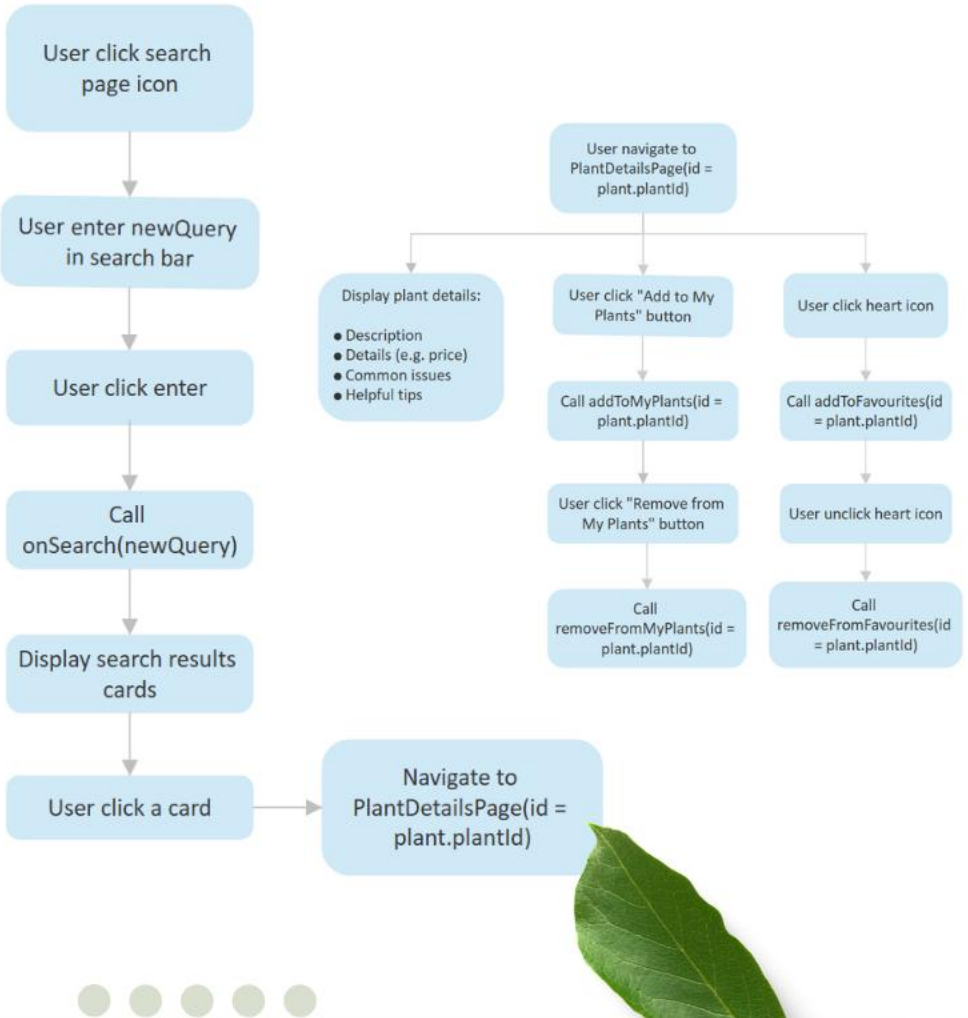
Search Page

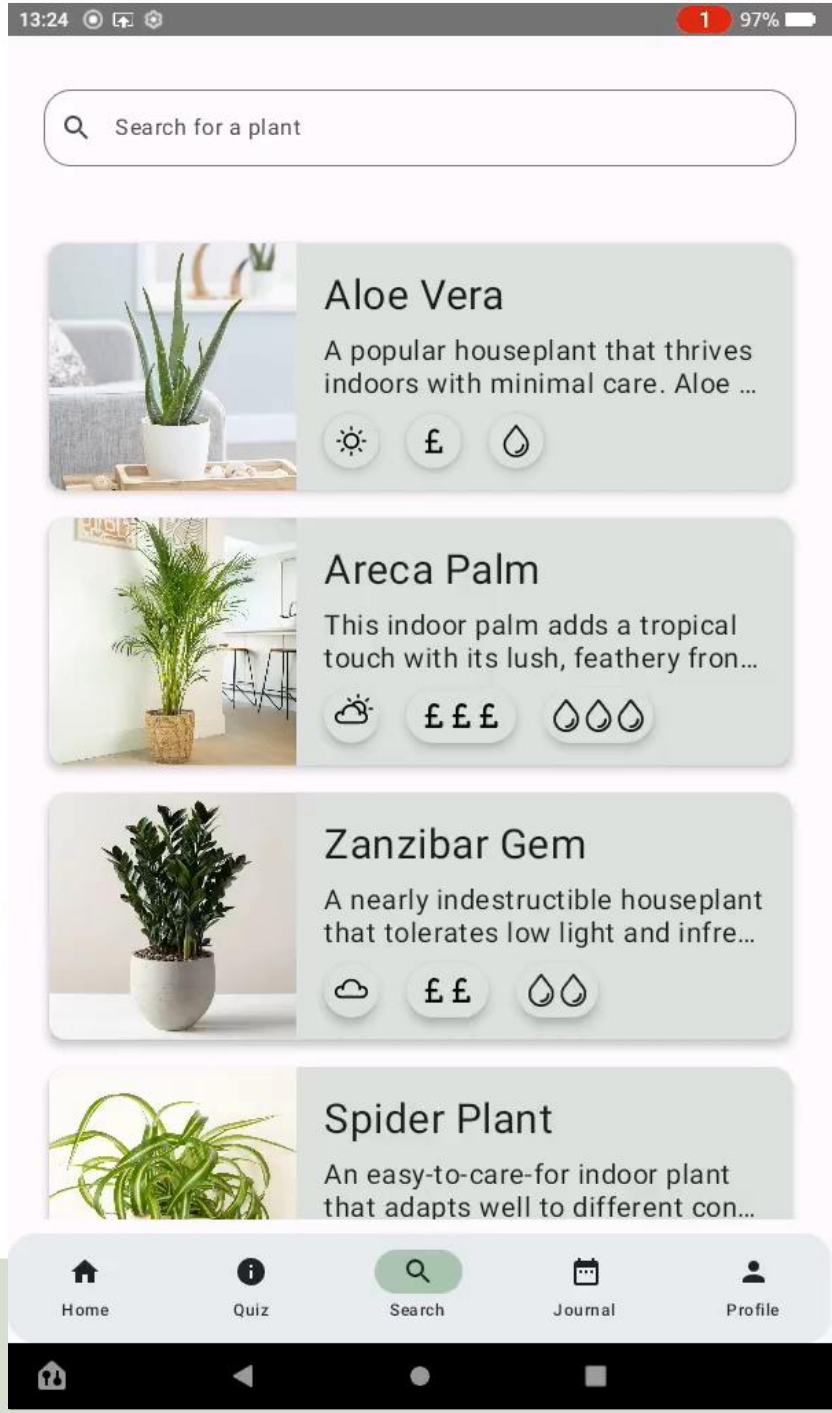
Key function:

- User is able to search for a plant and get more information about it
- User is able to add a plant to their list of owned plants or favourite it

Design considerations:

- Simplicity of layout, conciseness of text
- Consistent colour scheme, font, etc.
- Standard icons
- Readability (large text, etc.)





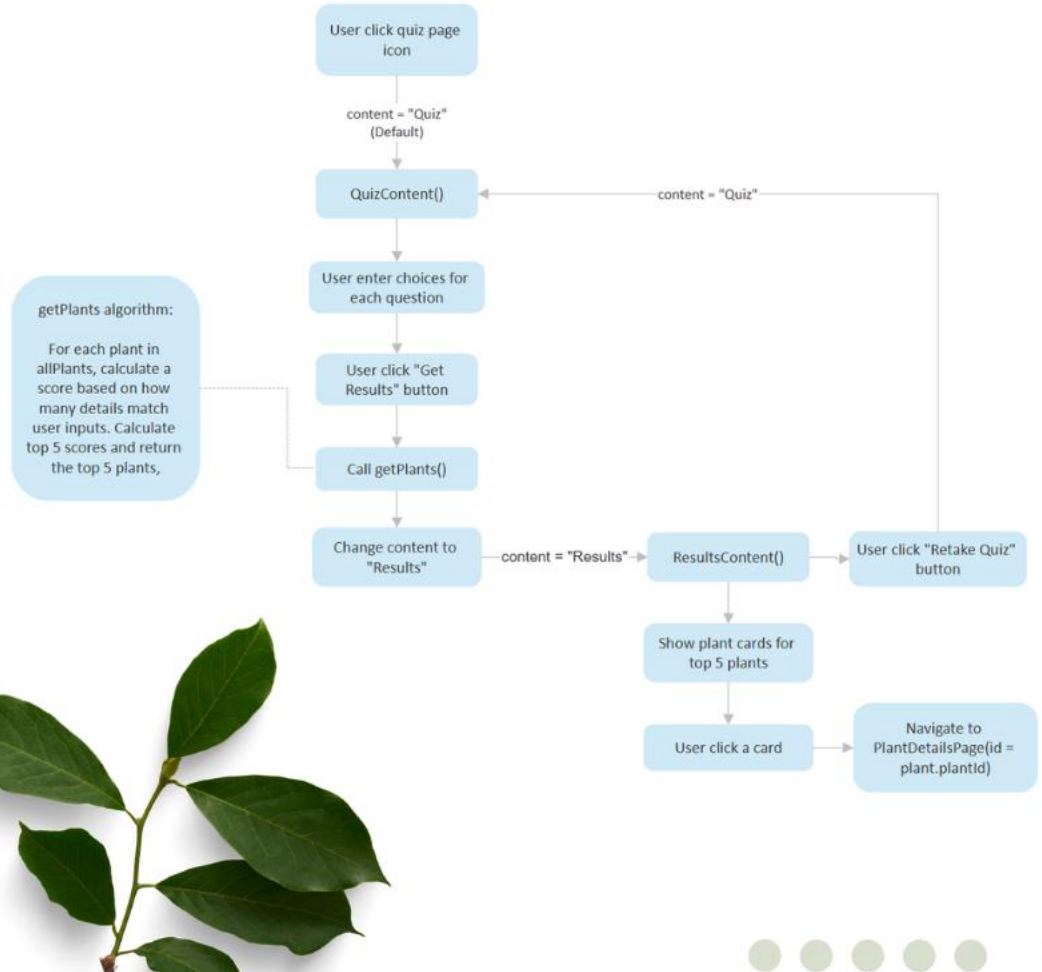
Quiz Page

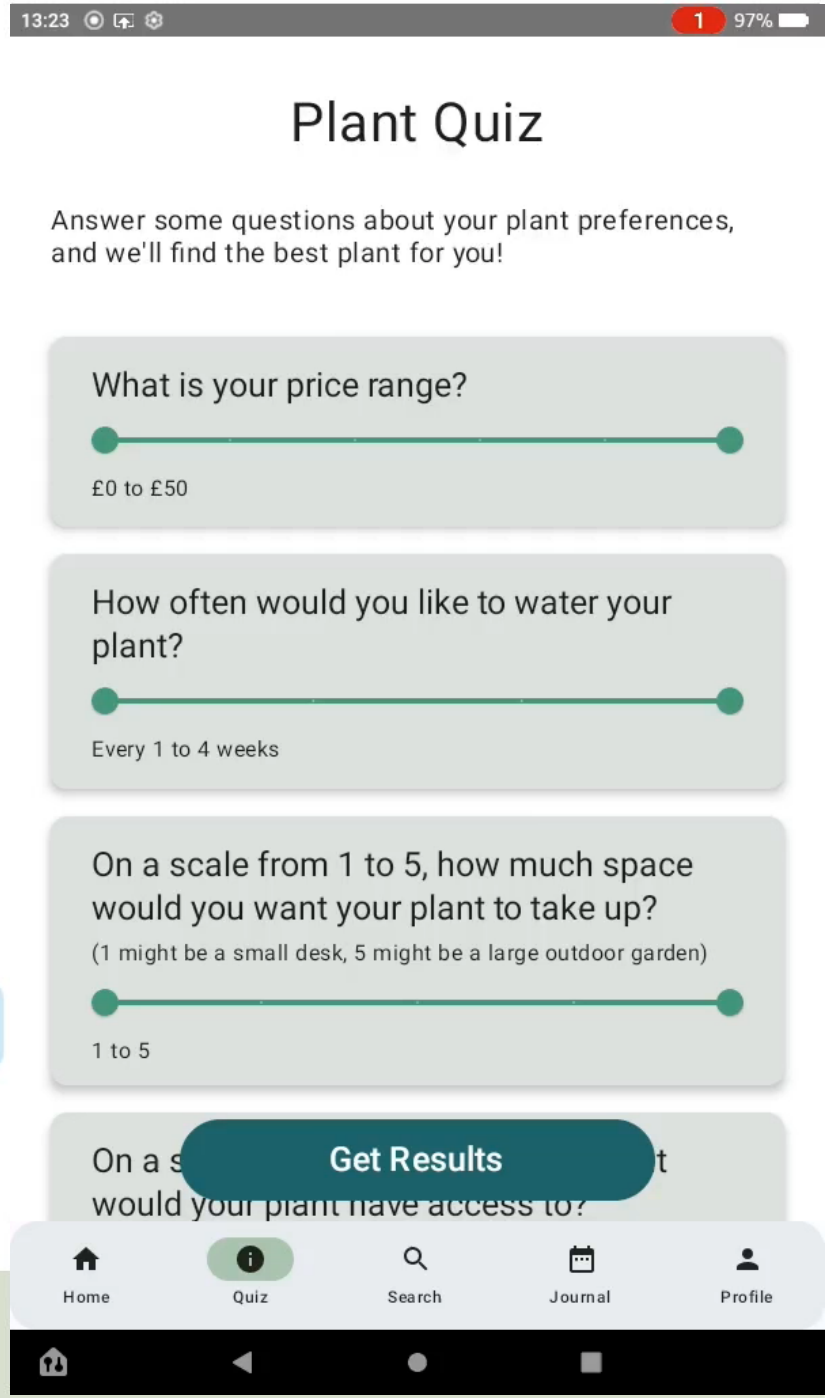
Key function:

- Advise users on what plant to get based on their personal preferences
- Provide some useful details for each plant

Design considerations:

- Simplicity of layout, conciseness of text
- Consistent colour scheme, font, etc.
- Standard icons
- Readability (large text, etc.)





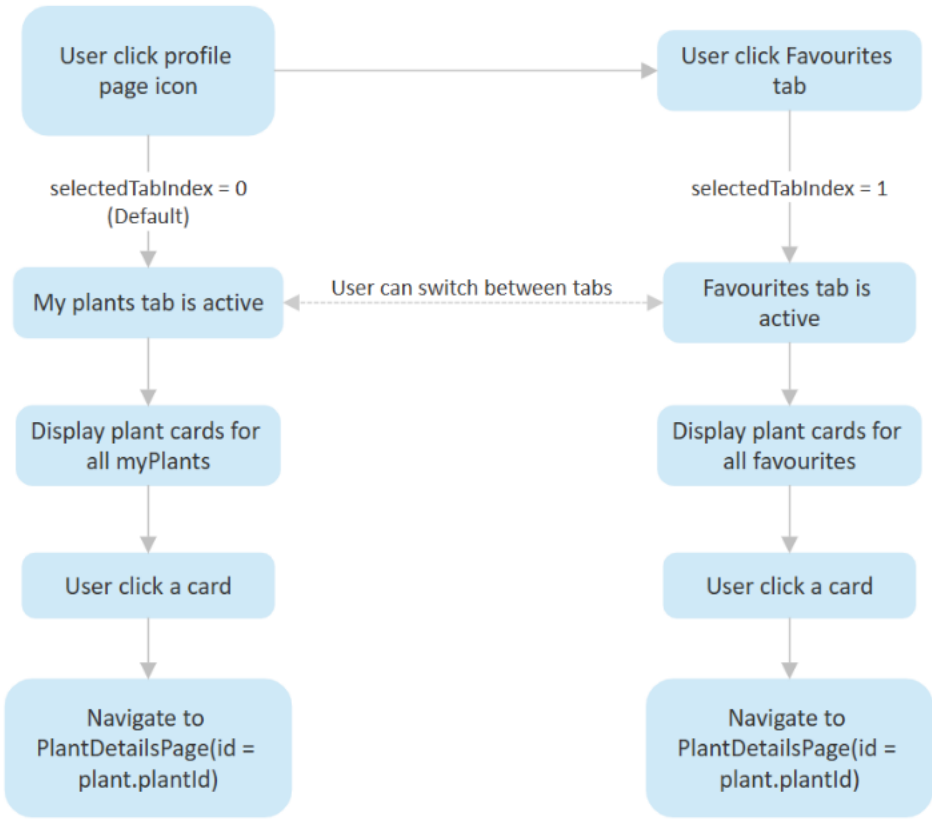
Profile Page

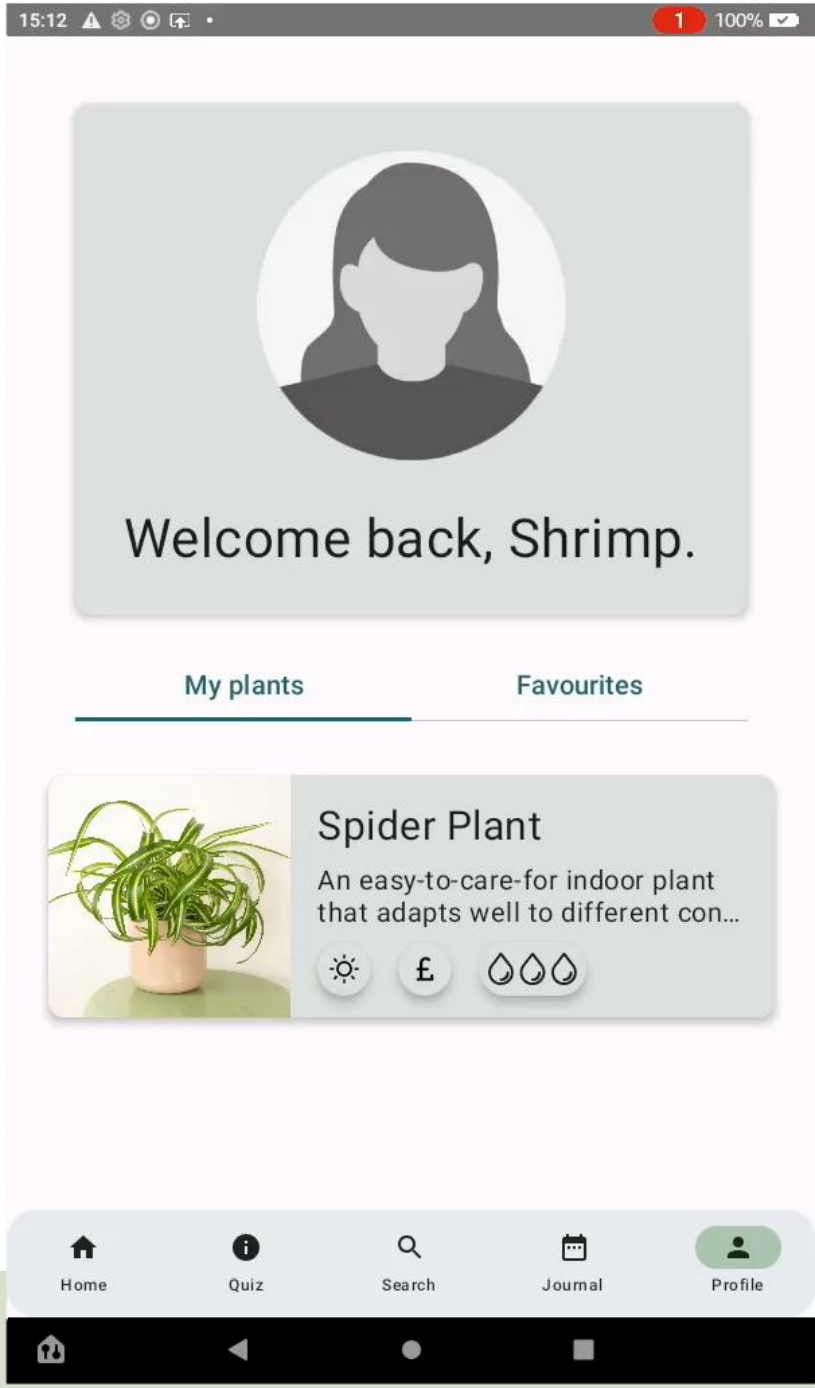
Key function:

- User is able to see their owned plants and favourited plants

Design considerations:

- Consistent colour scheme
- Readability (large text, etc.)
- Used same tab layout and plant cards as other pages for consistency
- Readability (large text, etc.)





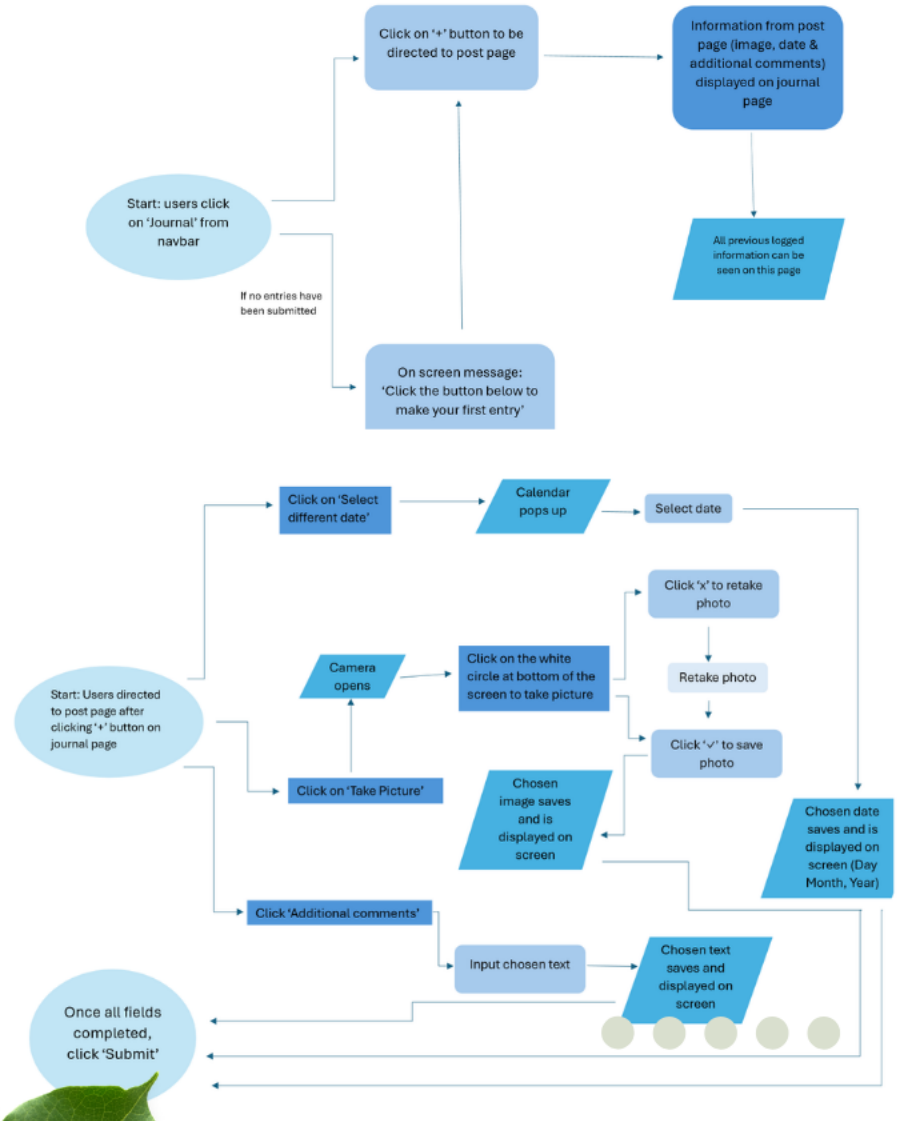
Journal / Post page

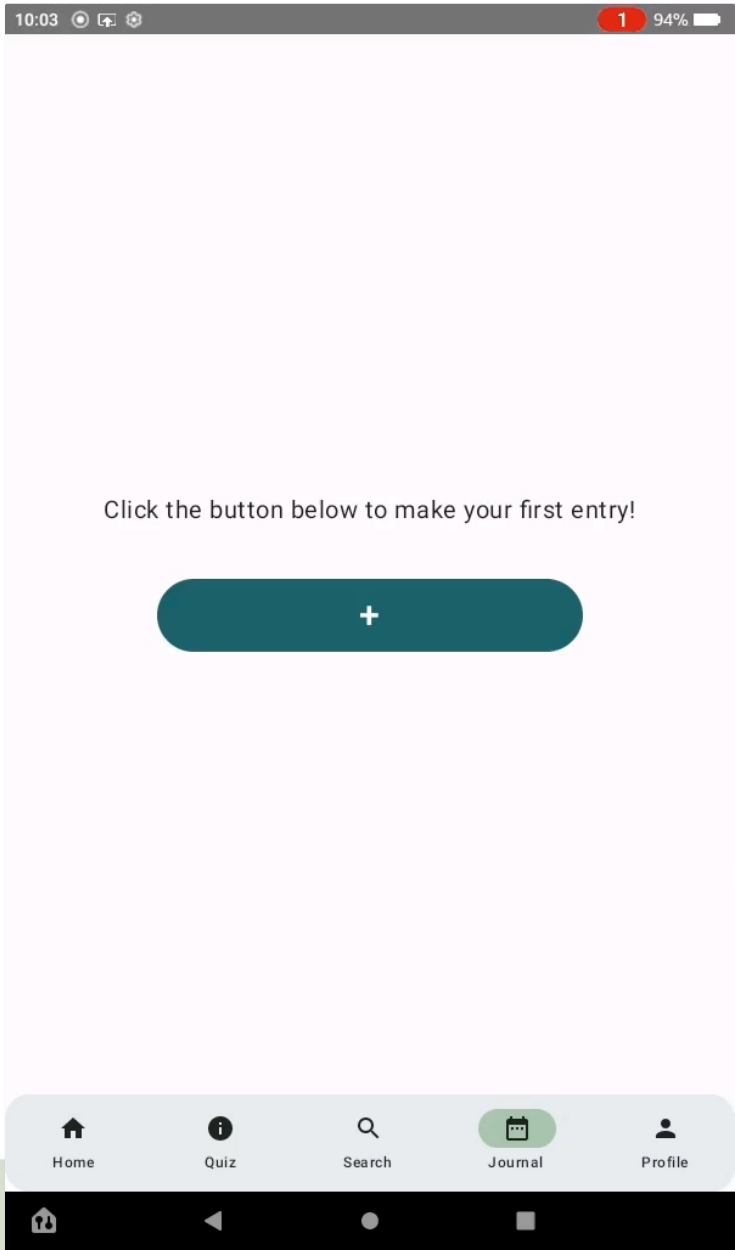
Key function:

- Users can log information about their plants
- Users can view information about previously uploaded plants
- Users can track the progress of their plant(s)

Design considerations:

- Standard icons for easy understandability
- Calendar instead of manual date entry for easier use
- Straightforward, understandable text





Camera Launcher/ Taking picture

```
fun CameraPreview(
    imageBitmap: Bitmap?,
    onLaunchCamera: () -> Unit
) {
```

```
Button(
    onClick = onLaunchCamera,
) {
    Text("Take Picture", fontSize = 20.sp)
}
```

```
imageBitmap?.let {
    Image(bitmap = it.asImageBitmap(), contentDescription = "Captured Image", modifier = Modifier.size(200.dp))
}
```

Displays the picture taken

By clicking the button, onLaunchCamera is called which opens the camera

Triggers the takePicture function, which opens the camera

```
// Camera launcher for taking picture
val cameraLauncher = rememberLauncherForActivityResult(ActivityResultContracts.TakePicturePreview()) { bitmap ->
    imageBitmap = bitmap
}

val darkBlue = colorResource(id = R.color.darkBlue)

// Camera permission logic
CameraPermission(
    onPermissionGranted = { hasPermission = true },
    onPermissionDenied = {
        Toast.makeText(context, "Camera permission denied", Toast.LENGTH_SHORT).show()
    }
)

// Handle Take Picture Button click
val takePicture = {
    if (hasPermission) {
        cameraLauncher.launch(null)
    } else {
        Toast.makeText(context, "Please grant camera permission.", Toast.LENGTH_SHORT).show()
    }
}
```

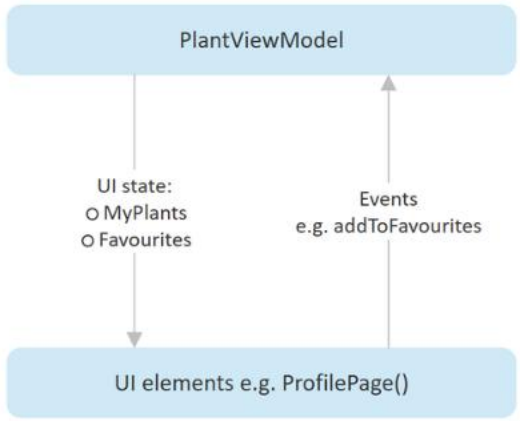
- Launches a contract to capture a picture through the device's camera
- Opens the camera and captures a preview image as a bitmap

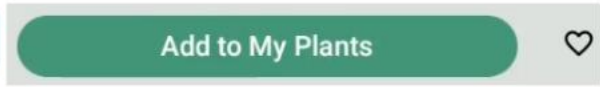
- Opens camera and takes picture if permission has been granted
- If permission denied, displays a toast message prompting user to grant permission

Updates permissions to true if permission is granted, enabling the "Take Picture"

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-feature android:name="android.hardware.camera.any"/>
```

ViewModel and OOP






```
// Class for data stored across different pages (i.e. ui state that can be accessed by entire app)
data class PlantUiState(  ShrOOmp *
    val favourites: List<String> = emptyList<String>(),
    val myPlants: List<UserPlant> = listOf(UserPlant(plantId = "4", waterWeek = 1, lastWateredDates
```

```
class PlantViewModel: ViewModel() {  ShrOOmp
    private val _uiState = MutableStateFlow(PlantUiState())
    val uiState: StateFlow<PlantUiState> = _uiState.asStateFlow()
```

```
// Add to favourites list
fun addToFavourites(plantId: String) {  ShrOOmp
    if (!_uiState.value.favourites.contains(plantId)) {
        val updatedFavourites = _uiState.value.favourites + plantId
        _uiState.update { currentState -> currentState.copy(favourites = updatedFavourites) }
    }
}
```

```
onClick = {
    if (!isMyPlants) {
        plantViewModel.addToMyPlants(plant)
    }
    else {
        plantViewModel.removeFromMyPlants(plant.plantId)
    }
```

```
data class QuizChoices (  ShrOOmp
    var priceStart: Int = 0,
    var priceEnd: Int = 50,
    var waterStart: Int = 1,
    var waterEnd: Int = 4,
    var spaceStart: Int = 1,
    var spaceEnd: Int = 5,
    var lightStart: Int = 1,
    var lightEnd: Int = 3,
    var toxicYn: Boolean? = null,
    var outdoor: Boolean? = null,
    var indoor: Boolean? = null,
)
```

For each plant in our dataset, a score is calculated based on how many of its details match the user's preferences. The top 5 results are obtained and displayed.

```
val allScores = MutableList( size: 11) { 0 }
// Calculates a score for each plant based on how many of
for (i in 0 ≤ .. ≤ 10) {
    val plant = allPlants[i]
    if (plant.price in priceStart ≤ .. ≤ priceEnd) {
        allScores[i] += 1
    }
    if (plant.waterWeek in waterStart ≤ .. ≤ waterEnd) {
        allScores[i] += 1
    }
    if (plant.space in spaceStart ≤ .. ≤ spaceEnd) {
        allScores[i] += 1
    }
    if (plant.light in lightStart ≤ .. ≤ lightEnd) {
        allScores[i] += 1
    }
    if (plant.toxic == toxic_yn) {
        allScores[i] += 1
    }
    if (plant.outdoor == outdoor) {
        allScores[i] += 1
    }
    else if (outdoor == true && indoor == true) {
        allScores[i] += 1 // When user has selected eithe
    }
}
// Gets the top 5 plants by index
val top5Indexes: List<Int> = allScores
    .withIndex()                        // Pair each score w
    .sortedByDescending { it.value } // Sort by score in
    .take( n: 5)                          // Take the top 5
    .map { it.index }                 // Extract the index
val top5Plants = mutableListOf<Plant>()
// Adds each of the top 5 plants to top5Plants<Plant> usi
for (index in top5Indexes) {
    top5Plants.add(allPlants[index])
}
return top5Plants
```

Calendar code - LocalDate




```
val today: LocalDate get() = LocalDate.now()
```

'LocalDate' -> holds actual date of current system being used

```
private fun toItemUiModel(date: LocalDate, isSelectedDate: Boolean) = CalendarUiModel.Date(
    isSelected = isSelectedDate,
    isToday = date.isEqual(today),
    date = date
)
```

Allows each date to be either selected or not selected / isSelected boolean is either true or false

Compares LocalDate with 'today'

Sets 'today' as current date value -> The date which is labelled as 'Today' on the screen is the current date.

User Research

Save the quiz answers so that they do not reset after leaving the page

```
data class QuizChoices (  ShrOOmp
    var priceStart: Int = 0,
    var priceEnd: Int = 50,
    var waterStart: Int = 1,
    var waterEnd: Int = 4,
    var spaceStart: Int = 1,
    var spaceEnd: Int = 5,
    var lightStart: Int = 1,
    var lightEnd: Int = 3,
    var toxicYn: Boolean? = null,
    var outdoor: Boolean? = null,
    var indoor: Boolean? = null,
)
```

```
data class PlantUiState(  ShrOOmp *
    val quizChoices: QuizChoices = QuizChoices(),
```

```
class PlantViewModel: ViewModel() {  ShrOOmp
    private val _uiState = MutableStateFlow(PlantUiState())
    val uiState: StateFlow<PlantUiState> = _uiState.asStateFlow()
```

```
fun updatePriceStart(priceStart: Int) {  ShrOOmp
    val newChoices = _uiState.value.quizChoices.copy(priceStart = priceStart)
    _uiState.update { currentState -> currentState.copy(quizChoices = newChoices) }
}
```

Welcome back message on profile page should match username entered on login page



Welcome back, Shrimp.

```
onClick = {
    if (errorMessage == "") {
        plantViewModel.addUser(username)
```

```
fun addUser(username: String) {  ShrOOmp
    _uiState.update { currentState -> currentState.copy(username = username) }
}
```

```
val uiState by plantViewModel.uiState.collectAsState()
```

```
Text(
    text: "Welcome back, ${uiState.username}.",
```

Add an "Either" option for the indoor vs outdoor question in the quiz page

Would you like an indoor or outdoor plant?

Indoor | Outdoor | Either

```
// Either button
Button(
    onClick = {
        plantViewModel.updateOutdoor( outdoor: true)
        plantViewModel.updateIndoor( indoor: true)
    },
```

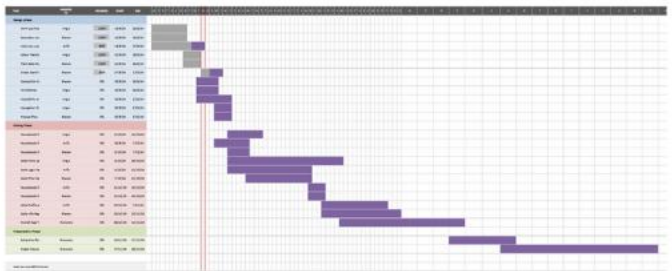
```
if (plant.outdoor == outdoor) {
    allScores[i] += 1
}
else if (outdoor == true && indoor == true) {
    allScores[i] += 1
}
```

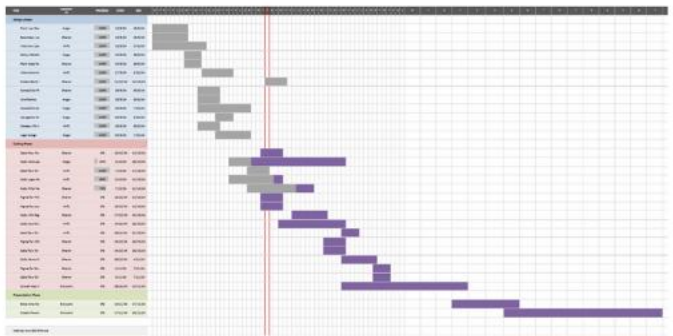
Collaboration

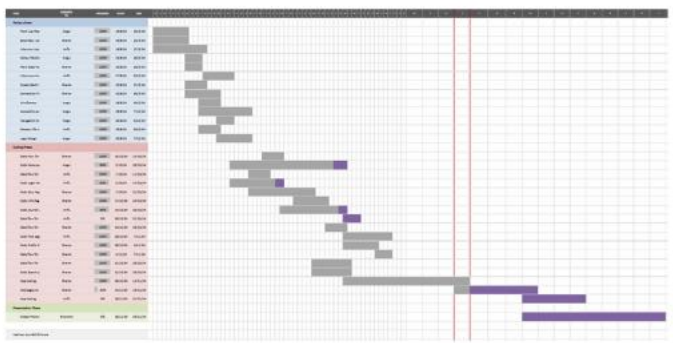
Spent at least 15 minutes at the start of each lesson filling in group journal and Gantt chart together

Tasks were split evenly and effectively so that we were able to cover more ground. E.g.

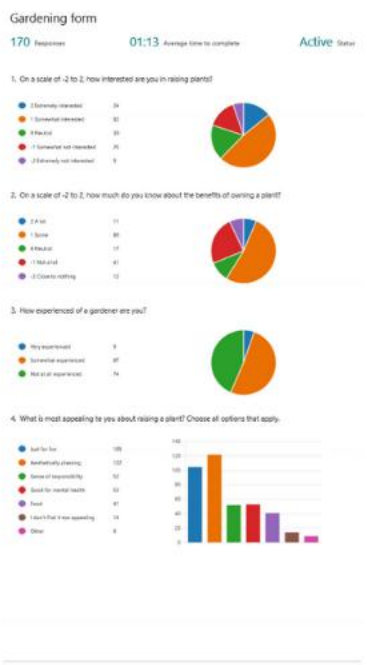
Communicated regularly on Teams, ensuring that everyone had always had a say, knew what was going on, and had a place to ask for assistance



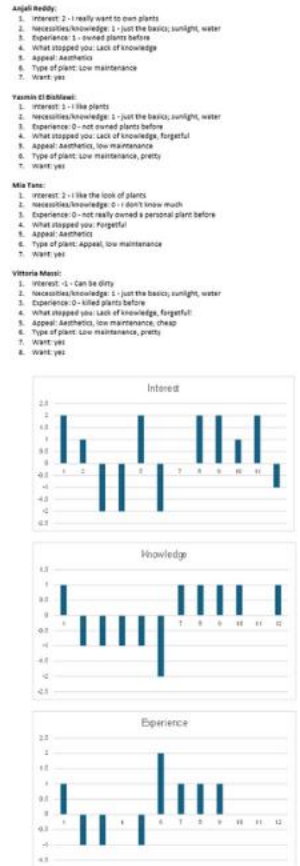




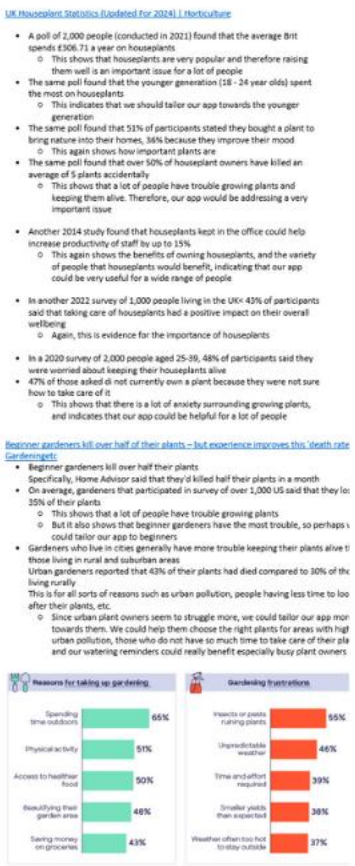
Angie: Form

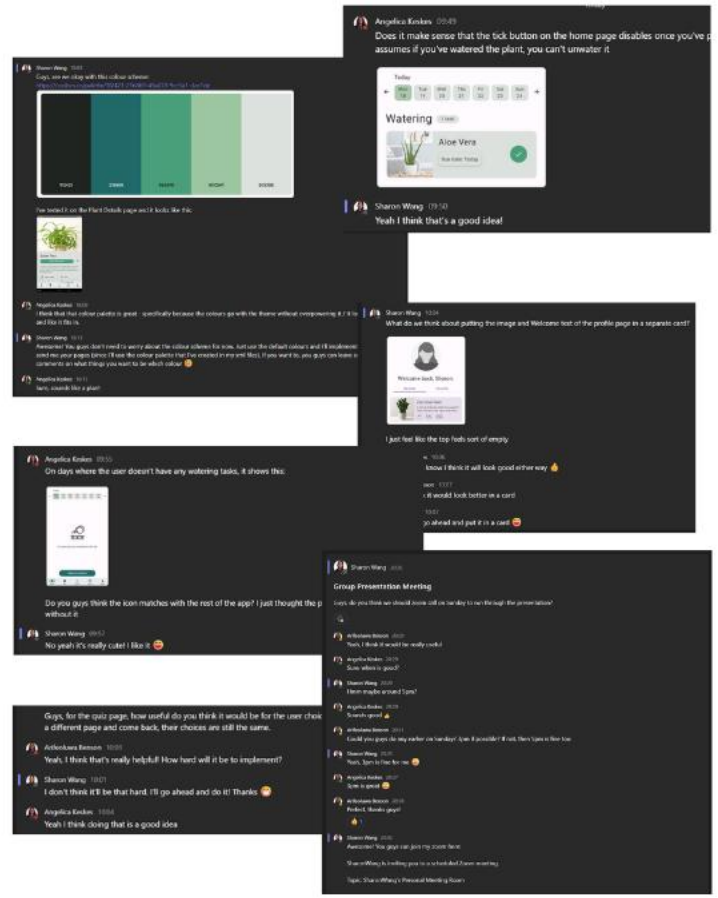


Arife: Interviews



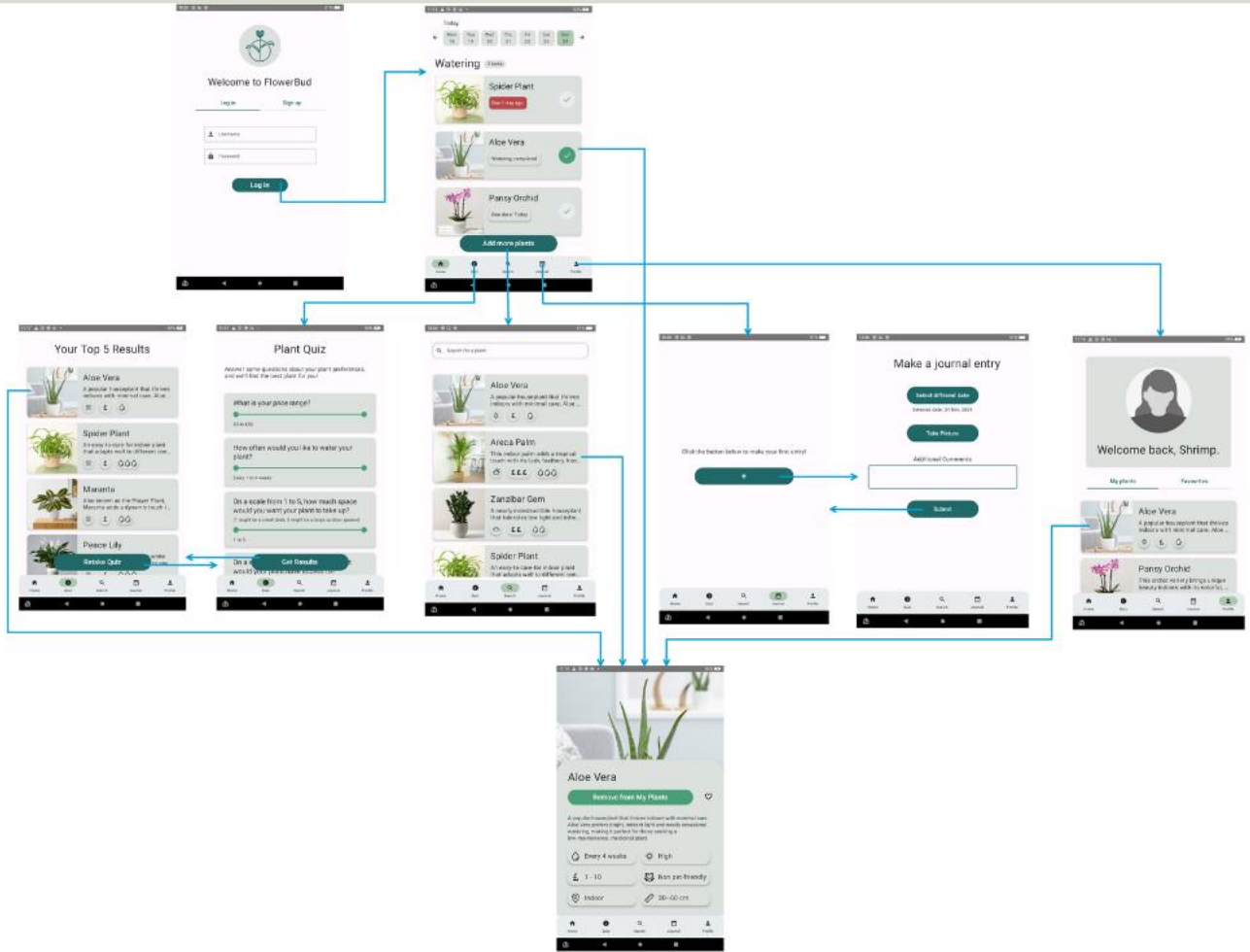
Sharon: Secondary

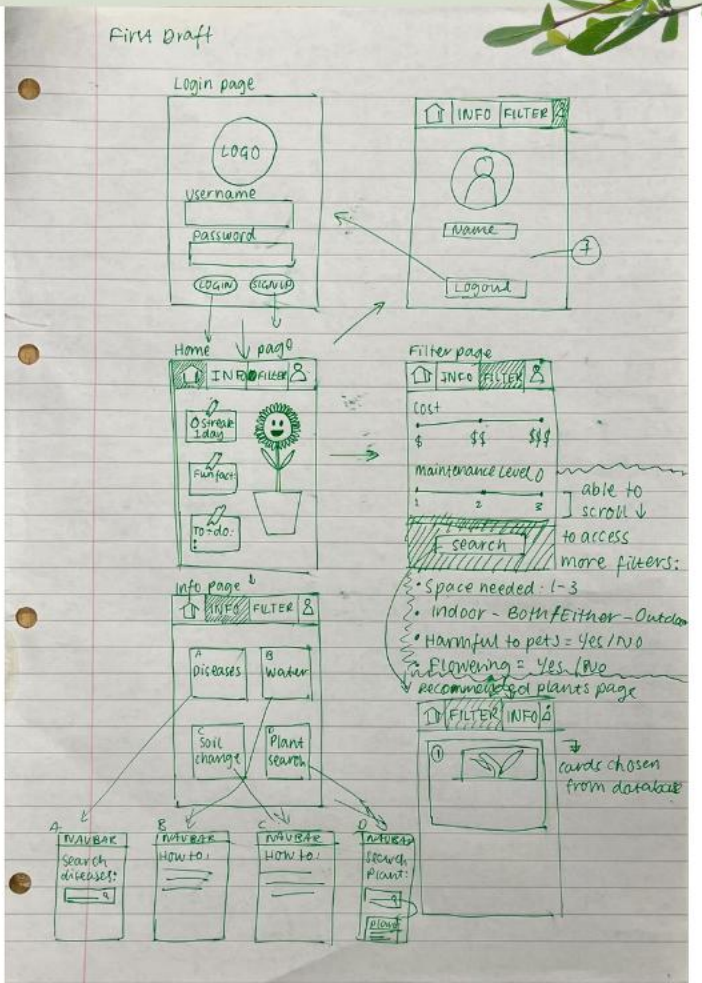




Challenges overcame

- Design choices
  - Changed a lot over time
- Coherence of pages





Improvements

Add more plants



- Find the perfect plant(s) for you

Fertilisation tasks



- Learn more about your plant(s)' needs

Socials page



- Join groups
  - Raise plants together
  - Share updates of plants
- Join events (e.g. dress up your plant)

THANK YOU!

